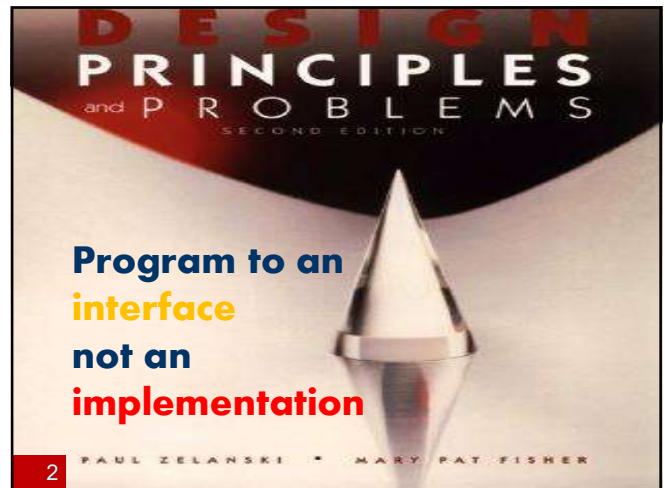


## Factory Pattern

1

1



2

2

### When you see "new", think "concrete".

Yes, when you use **new** you are certainly instantiating a concrete class, so that's definitely an implementation, not an interface. And it's a good question; you've learned that tying your code to a concrete class can make it more fragile and less flexible.

```
Duck duck = new MallardDuck();
```

We want to use interfaces to keep code flexible.

But we have to create an instance of a concrete class!

When you have a whole set of related concrete classes, often you're forced to write code like this:

```
Duck duck;

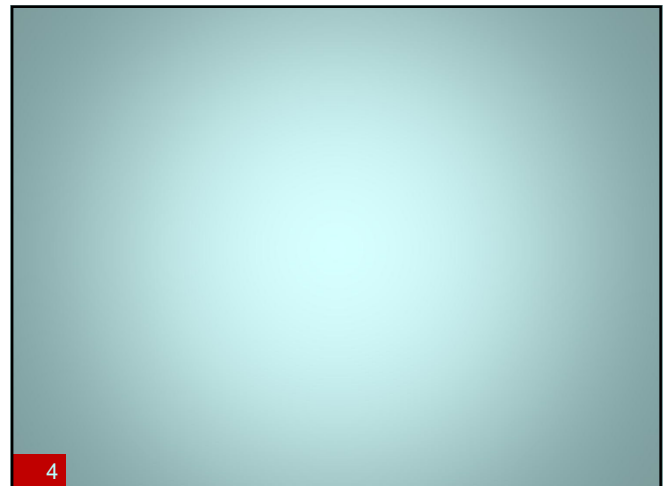
if (picnic) {
    duck = new MallardDuck();
} else if (hunting) {
    duck = new DecoyDuck();
} else if (inBathTub) {
    duck = new RubberDuck();
}
```

You try to use polymorphism but **new** force you to write such kind of code. a.k.a Program to implementation

We have a bunch of different duck classes, and we don't know until runtime which one we need to instantiate.

3

3



4

4

## Create objects

- You normally create objects with new operators
- However, new operators are not always good solution
- Its public interfaces can lead to coupling problem

5

5

Somewhere in the UI code that allow users to change the duck setting based in the background change

```
if (picnic) {
    duck = new MallardDuck();
} else if (hunting) {
    duck = new DecoyDuck();
} else if (inBathTub) {
    duck = new RubberDuck();
}
```

Somewhere in the code, when an alarm is off you want to alert the user with hundreds of annoying duck

```
when (alarm is off)
    for (i=0; i< 1000; i++)
        duck[i] = new StupidAnnoyingDuck();
```

Somewhere in the code, when you want surprise the birthday of a user

```
when (birthday is just now)
    duck = new AngelSingingDuck();
```

6

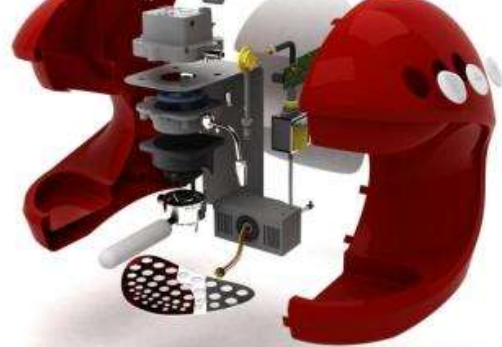
6

## NOTES

- It will be **reopened and examined** when the code should be changed
- This kind of code ends up in several parts of the application making maintenance and updates more difficult and error-prone

7

## Design Principle: Close for Modifications



8

8



OK. When we want to change  
"StupidAnnoyingDuck" to "PainInTheAssDuck"  
Is there a way not to pin down the code in  
the kernel code and open it for  
modifications?

Few Minutes !

9

9

## Close for Modifications

- The symptoms of violating this principle.
  - You do not need to open several places and change code as new concrete classes are added
- In addition, it is an violation of
  - Program to an interface, not an implementation

10

10

## Notes

- New operators involves with concrete object
- When you have code (**especially kernel code**) that makes use of lots of concrete classes, you are looking for trouble
- Instead of writing code with concrete objects, we have learned to write code with interfaces and using polymorphism

11

11

# HOW?

Identify the aspects that vary and separate them from what stays the same

12

12

## Baking a pizza

```
Pizza orderPizza() {
    Pizza pizza = new Pizza();

    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();
    return pizza;
}
```

```
Pizza orderPizza(String type) {
    Pizza pizza;

    if (type.equals("cheese")) {
        pizza = new CheesePizza();
    } else if (type.equals("greek")) {
        pizza = new GreekPizza();
    } else if (type.equals("pepperoni")) {
        pizza = new PepperoniPizza();
    }

    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();
    return pizza;
}
```

Can you identify the aspects that vary and separate them from what stays the same?

13

13

## Changes come!

```
Pizza orderPizza(String type) {
    Pizza pizza;

    if (type.equals("cheese")) {
        pizza = new CheesePizza();
    } else if (type.equals("greek")) {
        pizza = new GreekPizza();
    } else if (type.equals("pepperoni")) {
        pizza = new PepperoniPizza();
    } else if (type.equals("clam")) {
        pizza = new ClamPizza();
    } else if (type.equals("veggie")) {
        pizza = new VeggiePizza();
    }

    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();
    return pizza;
}
```

This code is NOT closed for modification. If the Pizza Shop changes its pizza offerings, we have to get into this code and modify it.

This is what varies. As the pizza selection changes over time, you'll have to modify this code over and over.

This is what we expect to stay the same. For the most part, preparing, cooking, and packaging a pizza has remained the same for years and years. So, we don't expect this code to change, just the pizzas it operates on.

14

14

## Encapsulating object creation

So now we know we'd be better off moving the object creation out of the `orderPizza()` method. But how? Well, what we're going to do is take the creation code and move it out into another object that is only going to be concerned with creating pizzas.

```
if (type.equals("cheese")) {
    pizza = new CheesePizza();
} else if (type.equals("pepperoni")) {
    pizza = new PepperoniPizza();
} else if (type.equals("clam")) {
    pizza = new ClamPizza();
} else if (type.equals("veggie")) {
    pizza = new VeggiePizza();
}
```

```
Pizza orderPizza(String type) {
    Pizza pizza;

    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();
    return pizza;
}
```

First we pull the object creation code out of the `orderPizza` method.

Then we place that code in an object that is only going to worry about how to create pizzas. If any other object needs a pizza created, this is the object to come to.

What's going to go here?

15

15

## Simple Pizza factory

Here's our new class, the `SimplePizzaFactory`. It has one job in life: creating pizzas for its clients.

First we define a `createPizza()` method in the factory. This is the method all clients will use to instantiate new objects.

```
public class SimplePizzaFactory {
    public Pizza createPizza(String type) {
        Pizza pizza = null;

        if (type.equals("cheese")) {
            pizza = new CheesePizza();
        } else if (type.equals("pepperoni")) {
            pizza = new PepperoniPizza();
        } else if (type.equals("clam")) {
            pizza = new ClamPizza();
        } else if (type.equals("veggie")) {
            pizza = new VeggiePizza();
        }

        return pizza;
    }
}
```

Here's the code we plucked out of the `orderPizza()` method.

This code is still parameterized by

16

16

## So simple?

### What is the benefits of doing this?

Q: What's the advantage of this? It looks like we are just pushing the problem off to another object.

There are no dumb questions.

PizzaShop class but is also a client of the factory.

So, by encapsulating the pizza creating in one class, we now have only one place to make modifications when the implementation changes.

Don't forget, we are also just about to remove the concrete instantiations from our client code!

Q: I've seen a similar design where a factory like this is defined as a static method. What is the difference?

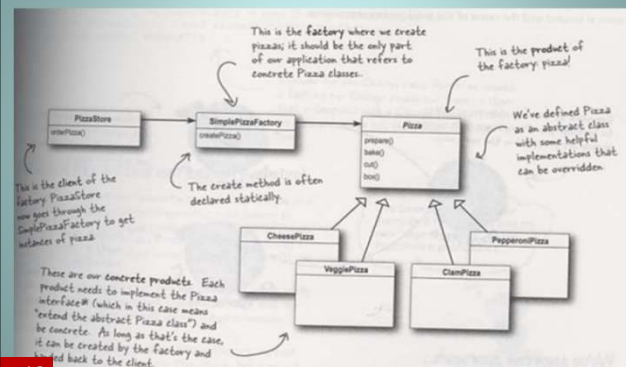
A: Defining a simple factory as a static method is a common technique and is often called a static factory. Why use a static method? Because you don't need to instantiate an object to make use of the create method. But remember it also has the disadvantage that you can't subclass and change the behavior of the create method.

### Don't worry, it is just a warm up

17

17

- The simple factory isn't actually a design pattern. It is more of a programming idiom but it is commonly used.



18

18



Make your code Independent of factory and extensive

```

public abstract class PizzaStore {
    public Pizza orderPizza(String type) {
        Pizza pizza;
        pizza = createPizza(type);
        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();
        return pizza;
    }
    abstract createPizza(String type);
}

```

PizzaStore is now abstract (see why below).

Now createPizza is back to being a call to a method in the PizzaStore rather than on a factory object.

All this looks just the same...

Now we've moved our factory object to this method.

Our "factory method" is now abstract in PizzaStore.

19

19

## Getting acquainted with factory

A factory method handles object creation and encapsulates it in a subclass. This decouples the client code in the superclass from the object creation code in the subclass.

A factory method may be parameterized (or not) to select among several variations of a product.

abstract Product factoryMethod(String type)

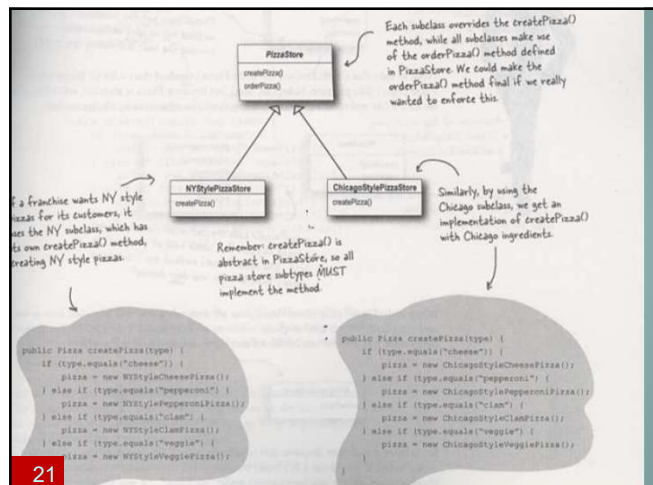
A factory method is abstract so the subclasses are counted on to handle object creation.

A factory method returns a Product that is typically used within methods defined in the superclass.

A factory method isolates the client (the code in the superclass, like orderPizza()) from knowing what kind of concrete Product is actually created.

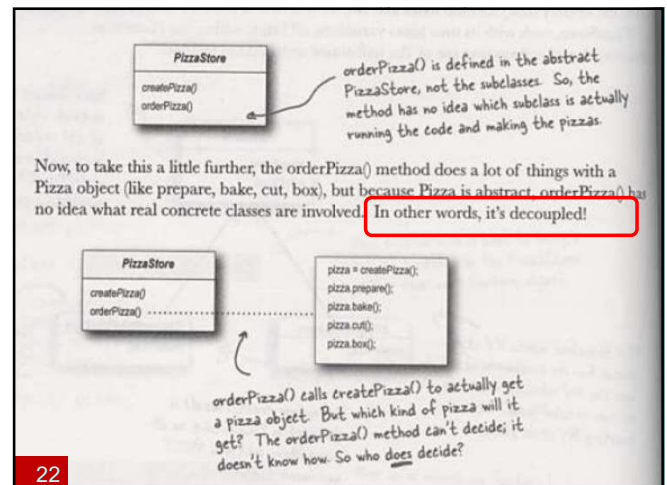
20

20



21

21



22

22

## Make a new Pizza store

createPizza() returns a Pizza, and the subclass is fully responsible for which concrete Pizza it instantiates.

The NYPizzaStore extends PizzaStore, so it inherits the orderPizza() method (among others).

We've got to implement createPizza(), since it is abstract in PizzaStore.

Here's where we create our concrete classes. For each type of Pizza we create the NY style.

```

public class NYPizzaStore extends PizzaStore {
    Pizza createPizza(String item) {
        if (item.equals("cheese")) {
            return new NYStyleCheesePizza();
        } else if (item.equals("veggie")) {
            return new NYStyleVeggiePizza();
        } else if (item.equals("clam")) {
            return new NYStyleClamPizza();
        } else if (item.equals("pepperoni")) {
            return new NYStylePepperoniPizza();
        } else return null;
    }
}

```

\* Note that the orderPizza() method in the superclass has no clue which Pizza we are creating; it just knows it can prepare, bake, cut, and box it!

23

23

## The test drive

```

public class PizzaTestDrive {
    public static void main(String[] args) {
        PizzaStore nyStore = new NYPizzaStore();
        PizzaStore chicagoStore = new ChicagoPizzaStore();

        Pizza pizza = nyStore.orderPizza("cheese");
        System.out.println("Ethan ordered a " + pizza.getName() + "\n");

        pizza = chicagoStore.orderPizza("cheese");
        System.out.println("Joel ordered a " + pizza.getName() + "\n");
    }
}

```

First we create two different stores.

Then use one store to make Ethan's order.

And the other for Joel's.

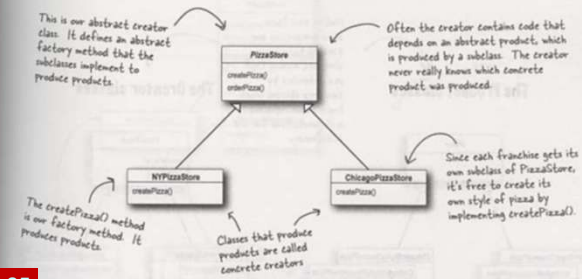
24

24

## The factory pattern

encapsulates object creation by letting subclasses decide which objects to create

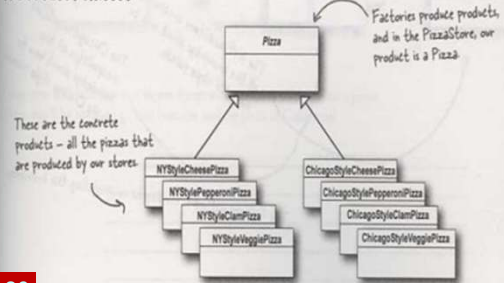
### The Creator classes



25

25

### The Product classes

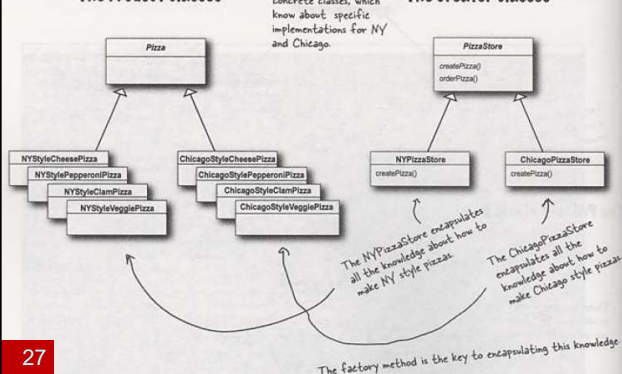


26

26

### The Product classes

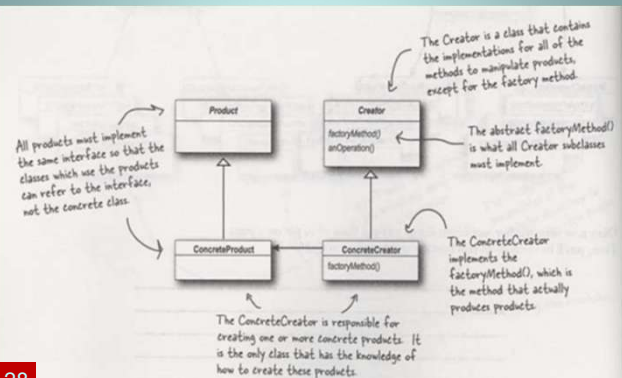
### The Creator classes



27

27

## The factory pattern defined



28

28

## The factory Method pattern

- Defines an interface for creating an object, but lets subclasses decide which class to instantiate. Factory method lets a class defer **instantiation** to subclasses

29

29

## Q&A

**With Questions**

**Q:** What's the advantage of the Factory Method Pattern when you only have one ConcreteCreator?

**A:** We implemented what is known as the parameterized factory method. It can make more than one object based on a parameter passed in, as you noticed. Often, however, a factory just produces one object and is not parameterized. Both are valid forms of the pattern.

**Q:** Your parameterized types don't seem "type-safe." I'm just passing in a String! What if I asked for a "CalmPizza"?

**A:** You are certainly correct and that would cause, what we call in the business, a "runtime error." There are several other more sophisticated techniques that can be used to make parameters more "type safe", or, in other words, to ensure errors in parameters can be caught at compile time. For instance, you can create objects that represent the parameter types, use static constants, or, in Java 5, you can use enums.

**Q:** I'm still a bit confused about the difference between Simple Factory and Factory Method. They look very similar, except that in Factory Method, the class that returns the pizza is a subclass. Can you explain?

**A:** They're similar, but used in different ways. Even though the implementation of each concrete store looks a lot like the SimplePizzaFactory, remember that the concrete stores are extending a class which has defined createPizza() as an abstract method. It is up to each store to define the behavior of the createPizza() method. In Simple Factory, the factory is another object that is composed with the PizzaStore.

30

30

## Continued

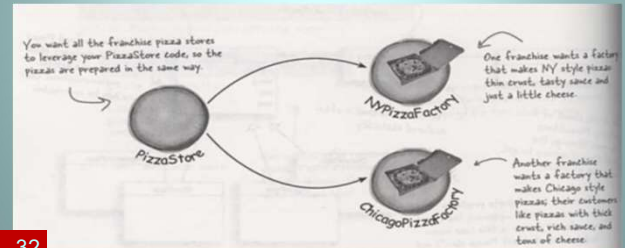
**A:** You're right that the subclasses do look a lot like Simple Factory, however think of Simple Factory as a one shot deal, while with Factory Method you are creating a framework that let's the subclasses decide which implementation will be used. For example, the orderPizza() method in the Factory Method provides a general framework for creating pizzas that relies on a factory method to actually create the concrete classes that go into making a pizza. By subclassing the PizzaStore class, you decide what concrete products go into making the pizza that orderPizza() returns. Compare that with SimpleFactory, which gives you a way to encapsulate object creation, but doesn't give you the flexibility of the Factory Method because there no way to vary the products you're creating.

31

31

## 讓我們將 pizza 店變成連鎖店

- Chicago pizza
- Pizzahut



32

32

factory can be instantiated to create pizza. But this is not good enough if you want to have some flexibility on different pizza making procedures

```

NYPizzaFactory nyFactory = new NYPizzaFactory();
PizzaStore nyStore = new PizzaStore(nyFactory);
nyStore.order("Veggie");

ChicagoPizzaFactory chicagoFactory = new ChicagoPizzaFactory();
PizzaStore chicagoStore = new PizzaStore(chicagoFactory);
chicagoStore.order("Veggie");

```

for making NY style pizzas

Then we create a PizzaStore and pass a reference to the NY factory.

and when we make pizzas, we get NY-styled pizzas.

Likewise for the Chicago pizza stores: we create a factory for Chicago pizzas and create a store that is composed with a Chicago factory. When we make pizzas, we get the Chicago flavored ones.

33

33

## The Dependency Inversion Principle

**Design Principle**  
 Depend upon abstractions. Do not depend upon concrete classes.

High level components should not depend on our low-level components

34

34

## The original PizzaStore

**Chan**

```

Pizza orderPizza(String type) {
    Pizza pizza;
    if (type.equals("cheese")) {
        pizza = new CheesePizza();
    } else if (type.equals("pepperoni")) {
        pizza = new PepperoniPizza();
    } else if (type.equals("clam")) {
        pizza = new ClamPizza();
    } else if (type.equals("veggie")) {
        pizza = new VeggiePizza();
    }
    pizza.prepare();
    pizza.bake();
    pizza.cut();
    return pizza;
}

```

This version of the PizzaStore depends on all those pizza objects, because it's creating them directly.

If the implementation of these classes change, then we may have to modify in PizzaStore.

Because any changes to the concrete implementations of pizza affects the PizzaStore, we say that the PizzaStore "depends on" the pizza implementations.

Every new kind of pizza we add creates another dependency for PizzaStore.

35

35

## Using factory method pattern

```

public abstract class PizzaStore {
    public Pizza orderPizza(String type) {
        Pizza pizza = createPizza(type);
        pizza.prepare();
        pizza.bake();
        pizza.cut();
        return pizza;
    }
    abstract createPizza(String type);
}

```

Pizza is an abstract class—an abstraction.

PizzaStore now depends only on Pizza, the abstract class.

The concrete pizza classes depend on the Pizza abstraction too, because they implement the Pizza interface (remember we're using "interface" in the general sense in the Pizza abstract class).

Our "factory method" is now abstract in PizzaStore.

36

36



## Guidelines of the principle (not necessary all the time)

The following guidelines can help you avoid OO designs that violate the Dependency Inversion Principle:

- No variable should hold a reference to a concrete class.
- No class should derive from a concrete class.
- No method should override an implemented method of any of its base classes.

If you use new, you'll be holding a reference to a concrete class. Use a factory to get around that!

If you derive from a concrete class, you're depending on a concrete class. Derive from an abstraction, like an interface or an abstract class.

If you override an implemented method, then your base class won't really be an abstraction to start with. Those methods implemented in the base class are meant to be shared by all your subclasses.

37

37

## A common scenario

```
class human {
    virtual void walk() { .... }
    virtual void breath() { ... }
};
class professor extends human {
    void writePaper();
};
class student extends human {
    void walk() { .... }
    void hangoutOnPTT();
    void watchGalson
};
```

```
// somewhere in the code
main() {
    human[] people ;
    human x = new student();
    people[0] = x ;
    human y = new professor();
    people [1] = y ;

    for each human p in people do {
        p.walk(); //OK
        p.breath() // OK
        p.handoutOnPTT() // not OK.
    }
```

this code also depends on a concrete class, not good. You need to ask yourself why you end up in this code

38

38

## Let's take a rest

39

39

## What is the difference between an abstract class and an implementation?

### Abstract class

- has methods that is virtual (C++) or abstract (java)
- May have attributes
- May have default method implementation
- If it has no default method implementation and attributes, it degrades to an interface

### interface

- Only methods

40

40

## 4 Principles of Good Design

Program to an **abstract class**  
not an **concrete class**

41

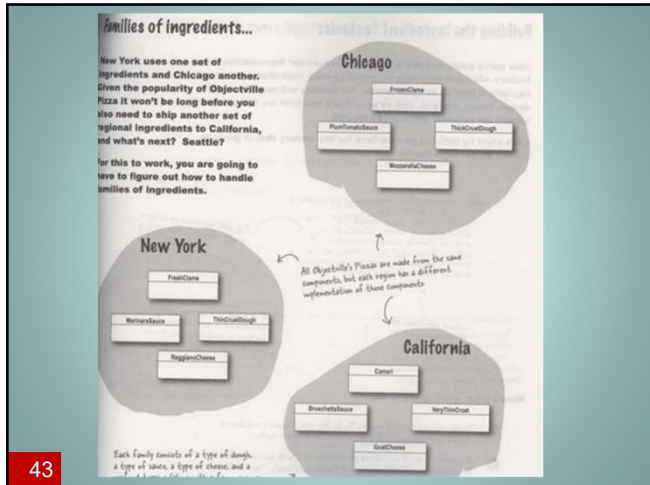
41

## What about toppings? (ingredients?)

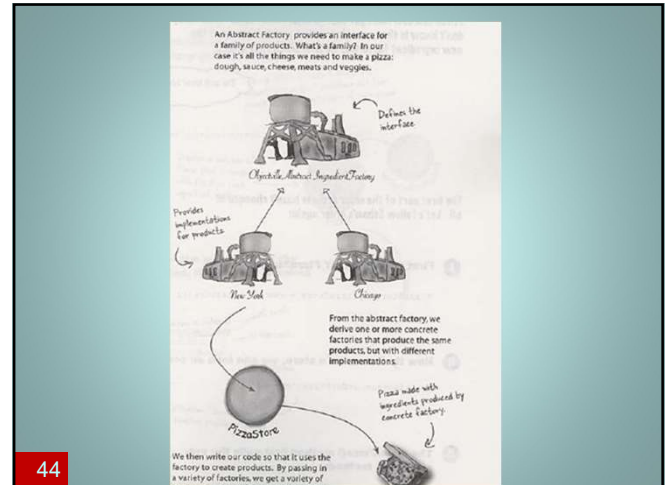
- Often factory method is enough for simple object creation. It relies on **inheritance**.
- When there is a need for creating object through **object composition**, you will need the **abstract factory method**

42

42



43



44

## Building the topping interface

```
public interface PizzaIngredientFactory {
```

```
    public Dough createDough();
    public Sauce createSauce();
    public Cheese createCheese();
    public Veggies[] createVeggies();
    public Pepperoni createPepperoni();
    public Clams createClam();
}
```

Lots of new classes here, one per ingredient.

For each ingredient we define a create method in our interface.

If we'd had some common "machinery" to implement in each instance of factory, we could have made this an abstract class instead.

45

```
public class NYPizzaIngredientFactory implements PizzaIngredientFactory {
    public Dough createDough() {
        return new ThinCrustDough();
    }
    public Sauce createSauce() {
        return new MarinaraSauce();
    }
    public Cheese createCheese() {
        return new ReggianoCheese();
    }
    public Veggies[] createVeggies() {
        Veggies veggies = { new Garlic(), new Onion(), new Mushroom(), new RedPepper() };
        return veggies;
    }
    public Pepperoni createPepperoni() {
        return new SlicedPepperoni();
    }
    public Clams createClam() {
        return new FreshClams();
    }
}
```

New York is on the coast; it gets fresh clams. Chicago has to settle for frozen.

The best sliced pepperoni. This is shared between New York and Chicago. Make sure you use it on the next page when you get to implement the Chicago factory yourself.

46

```
public abstract class Pizza {
    String name;
    Dough dough;
    Sauce sauce;
    Veggies veggies[];
    Cheese cheese;
    Pepperoni pepperoni;
    Clams clam;
    abstract void prepare();
    void bake() {
        System.out.println("Bake for 25 minutes at 500.");
    }
    void cut() {
        System.out.println("Cutting the pizza into diagonal slices.");
    }
    void box() {
        System.out.println("Place pizza in official PizzaStore box.");
    }
    void setName(String name) {
        this.name = name;
    }
    String getName() {
        return name;
    }
    public String toString() {
        // code to print pizza here
    }
}
```

Each pizza holds a set of ingredients that are used in its preparation.

We've now made the prepare method abstract. This is where we are going to collect the ingredients needed for the pizza, which of course will come from the ingredient factory.

Our other methods remain the same, with the exception of the prepare method.

47

## Make a new pizza

```
public class CheesePizza extends Pizza {
    PizzaIngredientFactory ingredientFactory;
    public CheesePizza(PizzaIngredientFactory ingredientFactory) {
        this.ingredientFactory = ingredientFactory;
    }
    void prepare() {
        System.out.println("Preparing " + name);
        dough = ingredientFactory.createDough();
        sauce = ingredientFactory.createSauce();
        cheese = ingredientFactory.createCheese();
    }
}
```

To make a pizza now, we need a factory to provide the ingredients. So each Pizza class gets a factory passed into its constructor, and it's stored in an instance variable.

Here's where the magic happens!

The prepare() method steps through creating a cheese pizza, and each time it needs an ingredient, it asks the factory to produce it.

48

47

48



## Make another kind of pizza

```
public class ClamPizza extends Pizza {
    PizzaIngredientFactory ingredientFactory;

    public ClamPizza(PizzaIngredientFactory ingredientFactory) {
        this.ingredientFactory = ingredientFactory;
    }

    void prepare() {
        System.out.println("Preparing " + name);
        dough = ingredientFactory.createDough();
        sauce = ingredientFactory.createSauce();
        cheese = ingredientFactory.createCheese();
        clam = ingredientFactory.createClam();
    }
}
```

ClamPizza also states an ingredient factory.

To make a clam pizza, the prepare method collects the right ingredients from its local factory.

49

49

## Open a new store

```
public class NYPizzaStore extends PizzaStore {
    protected Pizza createPizza(String item) {
        Pizza pizza = null;
        PizzaIngredientFactory ingredientFactory =
            new NYPizzaIngredientFactory();

        if (item.equals("cheese")) {
            pizza = new CheesePizza(ingredientFactory);
            pizza.setName("New York Style Cheese Pizza");
        } else if (item.equals("veggie")) {
            pizza = new VeggiePizza(ingredientFactory);
            pizza.setName("New York Style Veggie Pizza");
        } else if (item.equals("clam")) {
            pizza = new ClamPizza(ingredientFactory);
            pizza.setName("New York Style Clam Pizza");
        } else if (item.equals("pepperoni")) {
            pizza = new PepperoniPizza(ingredientFactory);
            pizza.setName("New York Style Pepperoni Pizza");
        }

        return pizza;
    }
}
```

The NY Store is equipped with a NY pizza ingredient factory. This will be used to produce the ingredients for all NY style pizzas.

We now pass each pizza the factory that should be used to produce its ingredients.

Look back one page and make sure you understand how the pizza and the factory work together!

For each type of Pizza, we instantiate a new Pizza and give it the factory it needs to get its ingredients.

50

50

## The steps revealed

**1 First we need a NY PizzaStore:**

```
PizzaStore nyPizzaStore = new NYPizzaStore();
```

Creates an instance of NYPizzaStore.

**2 Now that we have a store, we can take an order:**

```
nyPizzaStore.orderPizza("cheese");
```

the orderPizza() method is called on the nyPizzaStore instance.

**3 The orderPizza() method first calls the createPizza() method:**

```
Pizza pizza = createPizza("cheese");
```

51

51

The ingredient factory is chosen and instantiated in the PizzaStore and then passed into the constructor of each pizza.

```
Pizza pizza = new CheesePizza(nyIngredientFactory);
```

Creates a instance of Pizza that is composed with the New York ingredient factory.

**5 Next we need to prepare the pizza. Once the prepare() method is called, the factory is asked to prepare ingredients:**

```
void prepare() {
    dough = factory.createDough();
    sauce = factory.createSauce();
    cheese = factory.createCheese();
}
```

Thin crust  
Marinara  
Reggiano

For Ethan's pizza the New York ingredient factory is used, and so we get the NY ingredients.

52

52

## Abstract factory pattern

- Provides an interface for creating families of related or dependent objects without specifying their **concrete classes**

53

53

IMAX

世界上最高極致的  
視聽震撼觀影體驗

色彩鮮艷飽和  
音質清澈環繞  
身歷其境的臨場體驗

影像驚奇逼真  
畫面清晰震撼  
影像極大化的逼真視覺

數位 IMAX 強片推薦

全面啟動

7.16 (週五) IMAX 版本同步推出

LEONARDO DICAPRIO

YOUR MIND IS THE SCENE OF THE CRIME

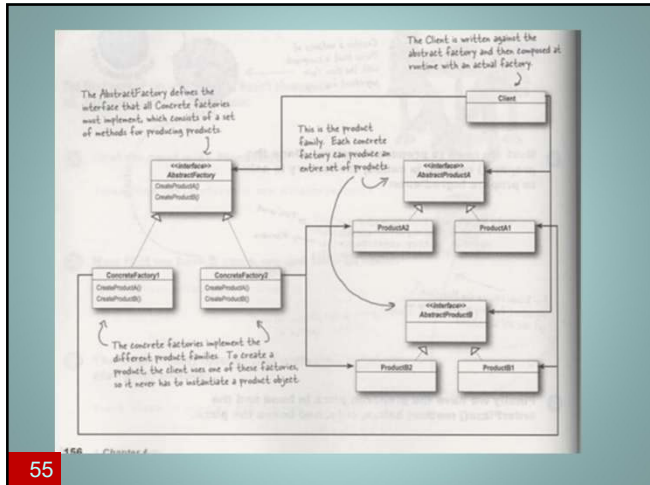
INCEPTION

FROM THE DIRECTOR OF THE DARK KNIGHT

EXPERIENCE IT JULY 16 IN THEATERS AND IMAX

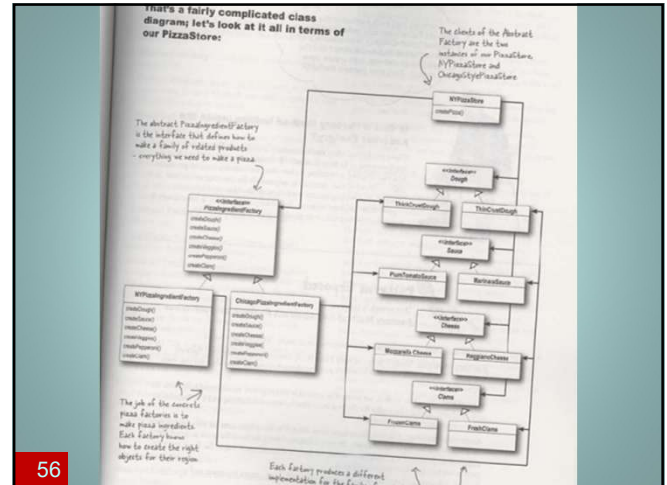
54

54



55

55



56

56

## Important Notes

- All factories encapsulate object creation.
- Simple Factory, while not a bona fide design pattern, is a simple way to decouple your clients from concrete classes.
- Factory Method relies on inheritance: object creation is delegated to subclasses which implement the factory method to create objects.
- Abstract Factory relies on object composition: object creation is implemented in methods exposed in the factory interface.
- All factory patterns promote loose coupling by reducing the dependency of your application on concrete classes.
- The intent of Factory Method is to allow a class to defer instantiation to its subclasses.
- The intent of Abstract Factory is to create families of related objects without having to depend on their concrete classes.
- The Dependency Inversion Principle guides us to avoid dependencies on concrete types and to strive for abstractions.
- Factories are a powerful technique for coding to abstractions, not concrete classes.

57

57

## How is the Abstract Factory different from the Factory Method pattern?

- An Abstract Factory is a class with several Factory methods. So basically, an Abstract Factory class has a collection of different Factory Methods to create different desired Concrete objects.
  - Factory Method**
    - The client expects an implementation of an interface or abstract class, but doesn't know exactly what concrete class the factory will return.
  - Abstract Factory**
    - Here, there is one more level of abstraction. The client does not even know what factory it's going to use. First, it gets a Factory and then it calls a Factory method. The following example will explain this.

58

58